?? Лабораторная работа № 2**

Первоначальная настройка git

Студент: Ань Сяоянь
Группа: НПИбд-01-25
**1132258191

1. Цель работы

Изучить идеологию и применение средств контроля версий, освоить умения по работе с git.
---

2. Выполнение работы

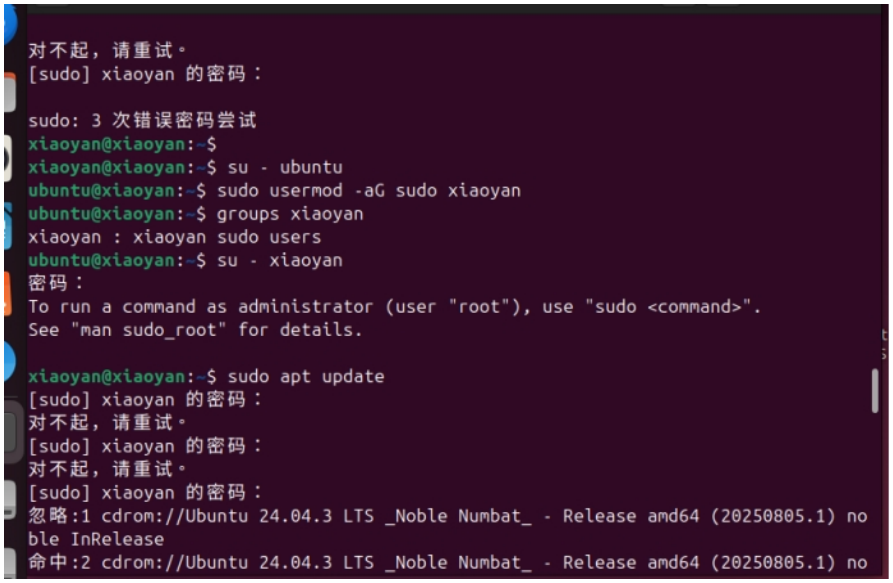
2.1 Установка git

Git был установлен с помощью менеджера пакетов apt:

```bash
sudo apt update
sudo apt install git -y
```

Результат:
```
git version 2.43.0
```

Скриншот 1 – Установка git и проверка версии



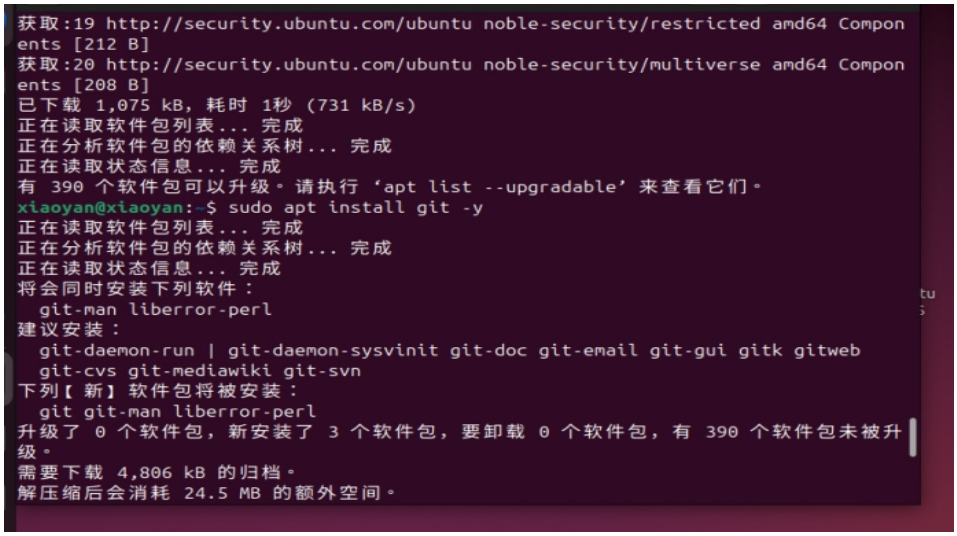
2.2 Базовая настройка git

Были выполнены следующие настройки:

Параметр	Значение	Команда
Имя пользователя	An xiaoyan	`git config --global user.name "An xiaoyan"`
Email	368829120@qq.com	`git config --global user.email "368829120@qq.com"`
UTF-8 в выводе	false	`git config --global core.quotepath false`
Ветка по умолчанию	master	`git config --global init.defaultBranch master`
Автоматический перевод строк	input	`git config --global core.autocrlf input`
Проверка перевода строк	warn	`git config --global core.safecrlf warn`

Проверка конфигурации:
```bash
git config --list
```

*Скриншот 2 – Настройка параметров



2.3 Создание SSH ключей

Были созданы SSH ключи по двум алгоритмам:
1. Ed25519 ключ:
```bash
ssh-keygen -t ed25519 -C "368829120@qq.com"
```

2. RSA ключ (4096 бит):
```bash
ssh-keygen -t rsa -b 4096 -C "368829120@qq.com"
```

Просмотр публичного ключа:
```bash
cat ~/.ssh/id_ed25519.pub
```

Результат:
```ssh-ed25519

AAAAC3NzaC1lZDI1NTE5AAAAIDmthKa806E6my8ai2uhNNeMNWU3Ghc5X0JWNKOVtNTn
368829120@qq.com```

*Скриншот 3 – Создание SSH ключей*

---

### **2.4 Настройка GitHub**

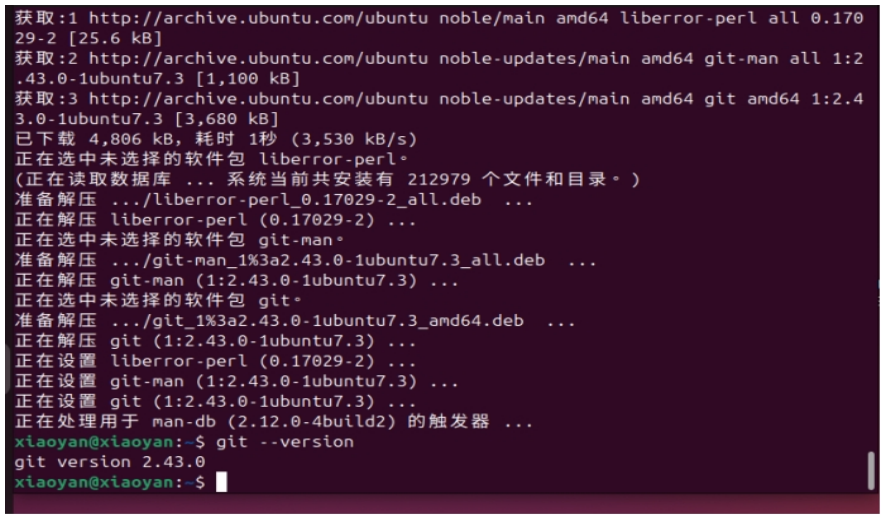
**1. Создание репозитория на GitHub:**
- Создан аккаунт на GitHub
- Создан публичный репозиторий `study_2025-2026_os-intro`

**2. Добавление SSH ключа в аккаунт GitHub:**
- Публичный ключ скопирован в буфер обмена
- Добавлен в настройках GitHub (Settings → SSH and GPG keys → New SSH key)

**3. Проверка подключения:**
```bash
ssh -T git@github.com
```

Результат:
```
Hi anxiaoyan999! You've successfully authenticated, but GitHub does not provide shell access.
```

Скриншот 4 – Проверка SSH подключения к GitHub
```



2.5 Создание локального репозитория

1. Создание структуры каталогов:
```bash
mkdir -p ~/work/study/2025-2026/OS-Intro
cd ~/work/study/2025-2026/OS-Intro
```

2. Клонирование репозитория:
```bash
git clone git@github.com:anxiaoyan999/study_2025-2026_os-intro.git os-intro
cd os-intro
```

Скриншот 5 – Клонирование репозитория

2.6 Добавление файлов и первый коммит
1. Создание файла COURSE:
```bash
echo "os-intro" > COURSE

**2. Добавление файла в индекс:**
```bash
git add .
```

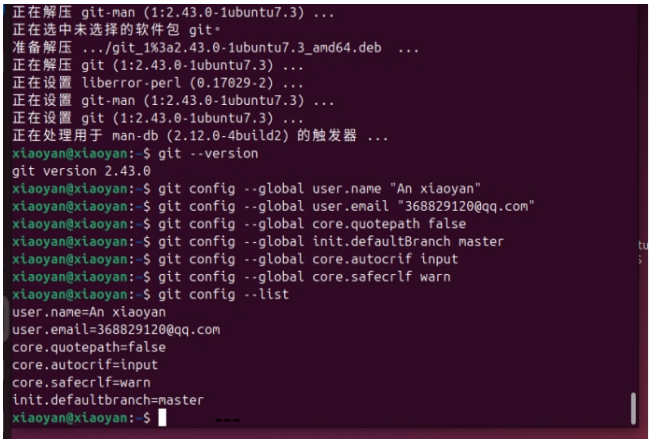
3. Создание коммита:
```bash
git commit -m "feat(main): add COURSE file"
```

Результат:
```
[main f544e23] feat(main): add COURSE file
 1 file changed, 1 insertion(+)
 create mode 100644 COURSE
```

4. Отправка изменений на GitHub:
```bash
git push
```

Результат:
```
枚举对象中：4，完成。
对象计数中：100% (4/4) ，完成。
写入对象中：100% (3/3) ，297 字节 | 297.00 KiB/s，完成。
To github.com:anxiaoyan999/study_2025-2026_os-intro.git
   a43c62a..f544e23 main -> main
```

Скриншот 6 – Добавление COURSE файла и отправка на GitHub
```



2.7 Просмотр результатов на GitHub

После выполнения всех операций на странице репозитория GitHub появился файл `COURSE`, подтверждающий успешную настройку и работу с git.
Скриншот 7 – Репозиторий на GitHub с файлом COURSE

---

3. Выводы

В ходе выполнения лабораторной работы были получены практические навыки:

- Установки и настройки системы контроля версий git
- Создания SSH ключей для безопасного подключения к GitHub
- Регистрации и настройки аккаунта на GitHub
- Создания локального и удаленного репозиториев
- Выполнения базовых операций git (clone, add, commit, push)
- Работы с ветками и файлами в репозитории

Все поставленные задачи выполнены, система готова к дальнейшей работе с git для ведения лабораторных работ по курсу.

---

4. Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (VCS) — это программные средства для управления изменениями в файлах и проектах. Они предназначены для:- Отслеживания истории изменений файлов
- Совместной работы нескольких разработчиков над одним проектом
- Возможности отката к предыдущим версиям
- Разрешения конфликтов при одновременном изменении файлов
- Хранения всех версий проекта в структурированном виде

2. Объясните понятия: хранилище, commit, история, рабочая копия

- **Хранилище (repository)** — место, где хранятся все файлы проекта и история их изменений
- **Commit** — фиксация изменений в репозитории с сохранением текущего состояния файлов
- **История (history)** — последовательность коммитов, показывающая все изменения проекта
- **Рабочая копия (working copy)** — локальная версия файлов проекта, с которой работает пользователь

3. Чем отличаются централизованные и децентрализованные VCS?

Централизованные VCS (например, CVS, Subversion):

- Единый сервер с основным репозиторием
- Пользователи получают рабочие копии
- Требуется подключение к серверу для большинства операций

Децентрализованные VCS (например, Git, Mercurial):
- Каждый пользователь имеет полную копию репозитория
- Большинство операций выполняется локально
- Можно работать без подключения к сети

4. Каковы основные задачи git?

- Управление версиями файлов
- Отслеживание изменений в проекте
- Обеспечение совместной работы разработчиков
- Создание и слияние веток разработки
- Синхронизация с удаленными репозиториями

5. Назовите основные команды git

Команда	Назначение
`git init`	Создание нового репозитория
`git clone`	Копирование удаленного репозитория
`git add`	Добавление изменений в индекс
`git commit`	Фиксация изменений
`git push`	Отправка изменений на удаленный репозиторий
`git pull`	Получение изменений из удаленного репозитория
`git status`	Просмотр состояния файлов
`git log`	Просмотр истории коммитов
`git branch`	Управление ветками
`git merge`	Слияние веток

6. Что такое ветви (branches) и зачем они нужны?

Ветви — это отдельные линии разработки в репозитории. Они позволяют:

- Работать над новыми функциями изолированно
- Исправлять ошибки не затрагивая основную разработку
- Экспериментировать с кодом без риска
- Параллельно разрабатывать несколько версий проекта

7. Как и зачем игнорировать некоторые файлы при commit?

Для игнорирования файлов используется файл `.gitignore`. В него записываются шаблоны файлов и каталогов, которые не должны попадать в репозиторий (например, временные файлы, объектные файлы, конфигурации среды). Это необходимо для:- Исключения служебных и временных файлов
- Уменьшения размера репозитория
- Предотвращения попадания конфиденциальных данных

---

5. Приложение
Список выполненных команд:

```
正在解压 git-man (1:2.43.0-1ubuntu7.3) ...
正在选中未选择的软件包 git。
准备解压 .../git_1%3a2.43.0-1ubuntu7.3_amd64.deb  ...
正在解压 git (1:2.43.0-1ubuntu7.3) ...
正在设置 liberror-perl (0.17029-2) ...
正在设置 git-man (1:2.43.0-1ubuntu7.3) ...
正在设置 git (1:2.43.0-1ubuntu7.3) ...
正在处理用于 man-db (2.12.0-4build2) 的触发器 ...
xiaoyan@xiaoyan:~$ git --version
git version 2.43.0
xiaoyan@xiaoyan:~$ git config --global user.name "An xiaoyan"
xiaoyan@xiaoyan:~$ git config --global user.email "368829120@qq.com"
xiaoyan@xiaoyan:~$ git config --global core.quotepath false
xiaoyan@xiaoyan:~$ git config --global init.defaultBranch master
xiaoyan@xiaoyan:~$ git config --global core.autocrlf input
xiaoyan@xiaoyan:~$ git config --global core.safecrlf warn
xiaoyan@xiaoyan:~$ git config --list
user.name=An xiaoyan
user.email=368829120@qq.com
core.quotepath=false
core.autocrlf=input
core.safecrlf=warn
init.defaultbranch=master
xiaoyan@xiaoyan:~$
```

```
sudo apt update
sudo apt install git -y
git --version
git config --global user.name "An xiaoyan"
git config --global user.email "368829120@qq.com"
git config --global core.quotepath false
git config --global init.defaultBranch master
git config --global core.autocrlf input
git config --global core.safecrlf warn
git config --list
ssh-keygen -t ed25519 -C "368829120@qq.com"
ssh-keygen -t rsa -b 4096 -C "368829120@qq.com"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
mkdir -p ~/work/study/2025-2026/OS-Intro
cd ~/work/study/2025-2026/OS-Intro
git clone git@github.com:anxiaoyan999/study_2025-2026_os-intro.git os-intro
cd os-intro
echo "os-intro" > COURSE
git add .
git commit -m "feat(main): add COURSE file"
git push```
```

```
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro$ cd os-intro
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$ echo "os-intro" > COUR
SE
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$ git add
没有指定文件，也没有文件被添加。
提示：也许您想要执行 'git add .'？
提示：运行下面的命令来关闭本消息
提示："git config advice.addEmptyPathspec false"
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$ git add .
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$ git commit -m "feat(ma
in): add COURSE file"
[main f544e23] feat(main): add COURSE file
 1 file changed, 1 insertion(+)
 create mode 100644 COURSE
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$ git push
枚举对象中：4，完成.
对象计数中：100% (4/4)，完成.
使用 2 个线程进行压缩
压缩对象中：100% (2/2)，完成.
写入对象中：100% (3/3)，297 字节 | 297.00 KiB/s，完成.
总共 3（差异 0），复用 0（差异 0），包复用 0
To github.com:anxiaoyan999/study_2025-2026_os-intro.git
   a43c62a..f544e23  main -> main
xiaoyan@xiaoyan:~/work/study/2025-2026/OS-Intro/os-intro$
```